

# Saga Orchestrator Design Document

Purpose: Coordinate the distributed booking transaction. The Saga Orchestrator owns workflow state only; it never owns business entities such as Booking, Payment or Seat.

## Responsibilities

- Coordinate distributed transaction
- Execute workflow
- Trigger compensation
- Retry failed steps
- Persist saga state

## Owned Entity

SagaInstance

## Source of Truth

Workflow execution status and progress.

## High-Level Architecture

Booking Service | BookingCreated | v Saga Orchestrator | | | +----->  
Inventory (Hold Seat) | +-----> Payment (Process Payment) | v  
Booking Confirmed / Cancelled | v Notification Service

## Saga Flow

BookingCreated → HoldSeatCommand → SeatHeld → ProcessPaymentCommand → PaymentSucceeded → ConfirmBookingCommand → BookingConfirmed. On PaymentFailed or SeatHoldFailed, execute ReleaseSeatCommand and CancelBookingCommand.

## Database Schema (saga\_db)

SagaInstance ----- saga\_id (PK) saga\_type  
correlation\_id (booking\_id) current\_step status retry\_count last\_error  
started\_at updated\_at completed\_at

## State Machine

STARTED → HOLDING\_SEAT → PAYMENT\_IN\_PROGRESS → CONFIRMING\_BOOKING → COMPLETED  
Failure Path: HOLD\_FAILED or PAYMENT\_FAILED → COMPENSATING → FAILED

## Kafka Events Consumed

BookingCreated, SeatHeld, SeatHoldFailed, PaymentSucceeded, PaymentFailed

## Kafka Commands/Events Published

HoldSeatCommand, ReleaseSeatCommand, ProcessPaymentCommand, ConfirmBookingCommand, CancelBookingCommand

## Retry Strategy

Retry transient failures using exponential backoff. Persist `retry_count`. Move to FAILED after maximum retries and execute compensation.

## Business Rules

Saga owns only workflow state. Every step is idempotent. Correlation is maintained using `booking_id`. Compensation must be safe to execute multiple times.

## Reliability

Dedicated `saga_db`, Kafka-based orchestration, idempotent consumers, observability with logs, metrics and tracing.